COMPILER DESIGN

LAB MANUAL – ANSWERED EDITION

(22CS2235F)

Academic Year: 2026–27

Student Name: G. Sirisha

Student ID: 2420030091

Contents

1. Removing white spaces and comments using LEX

2. Lexer tokenization focusing on keywords using LEX

3. Automating lexical analyzer generation using LEX

4. Elimination of left recursion from CFG

5. Computing FIRST set

6. Computing FOLLOW set

7. Construction of LL(1) parsing table

8. Shift-reduce parser

9. LR parser

10. Generating and interpreting quadruples

11. Code optimization technique

12. Intermediate code generation

13. Basic error detection and reporting system

SESSION 1: Removing white spaces and comments using LEX

Aim: To remove whitespace and both C comment forms from source text using lexical rules.

Pre-lab answers

1a. State diagram for whitespace. Use a start state q0. On a space/tab/newline, remain in q0 (or move to q1 while collecting a whitespace run); on any non-whitespace character, stop the token and return that character to the input. In LEX, [\t\n]+ matches the complete whitespace run.

1b. State diagram for C comments. For // comments: after /, a second / enters line-comment state; consume characters until newline, then return to INITIAL. For /* */: after / then *, enter block-comment state; consume until the sequence */ is found, then return to INITIAL.

2. Running LEX on Windows. Install a toolchain such as MSYS2/WinFlexBison or use WSL Ubuntu. Save rules as file.l; run flex file.l; compile lex.yy.c with GCC and the Flex library; execute the generated program. Add tool directories to PATH if commands are not found.

3. LEX installation. On Ubuntu/Colab: sudo apt update; sudo apt install flex bison gcc. Verify with flex --version and bison --version. On Windows, install MSYS2, install GCC and Flex/Bison packages, then run commands in the MSYS2 terminal.

4. NFA for (((00)*(11)) ∪01)*. Build a union using ε-branches from a new start to two branches: branch A recognizes (00)*(11), branch B recognizes 01. Connect both branch ends to a new accepting state; add ε from that accepting state back to the new start and make the new start accepting to implement Kleene star.

5a. Strings not containing 00. Regular expression: (1|01)*(ε|0). This allows isolated zeros only, never two consecutive zeros.

5b. At least three 1s. (0|1)*1(0|1)*1(0|1)*1(0|1)*.

5c. Characters in equal consecutive pairs. (00|11)*. It generates ε and any concatenation of 00 and 11 blocks.

In-lab: procedure and implementation

1. Open Google Colab or a C development environment.
2. Create a new code cell / source file and paste the complete program below.
3. Compile or run using the commands shown; enter the sample input.
4. Compare the displayed result with the sample output.

Execution environment: LEX / Flex with GCC

Complete program

```
%{
#include <stdio.h>
%}
%%
[ \t\r\n]+                          ;
"//".*                              ;
"/*"([^*]|\*+[^*/])*\*+"/"          ;
.                                   { ECHO; }
%%
int main(void) { yylex(); return 0; }
int yywrap(void) { return 1; }
```

Compile / run

```
flex program.l
gcc lex.yy.c -o scanner -lfl
```

```
./scanner
```

Output preview (use the sample input shown)

```
Input:  int main() { /* block */ printf("Hi"); // line comment
 }
Output: intmain(){ printf("Hi");  }
```

Result: The program demonstrates the stated compiler-design concept and produces the expected output for the sample input.

Post-lab answers

LEX program to identify integer/real constants. Use [0-9]+ for integers and [0-9]+\.[0-9]+ for real constants; print the matching lexeme and classify it. A sign is normally a separate unary operator.

LEX program to validate an identifier. Use [A-Za-z_][A-Za-z0-9_]*. This accepts a letter or underscore first, followed by letters, digits or underscores. Add keyword rules before the identifier rule.

Viva-voce answers

Q1. Define regular expression and applications. A regular expression is a notation for describing a regular language using union, concatenation and Kleene star. Example: a(b|c)*. Applications include lexical-analyzer token definitions, search patterns, input validation and text processing.

Q2. Compiler vs interpreter. A compiler translates a source program into target code before execution; an interpreter executes source or an intermediate representation step by step. Compiled programs often run faster; interpreters support interactive execution and easier debugging.

Q3. NFA vs DFA. A DFA has exactly one transition for each state/input symbol and no ε-moves. An NFA may have multiple or zero transitions and ε-moves. Both recognize exactly the regular languages.

Q4. Compiler pass vs phase. A phase is a logical function such as lexical, syntax or semantic analysis. A pass is one traversal of the source or intermediate representation. One pass may perform several phases; a phase may span multiple passes.

Q5. Front-end vs back-end; why separate scanning and parsing? The front-end analyzes source and builds an intermediate representation; the back-end optimizes and generates target code. Lexical analysis groups characters into tokens, while parsing checks grammatical structure. Separation simplifies design and reuse; merging can reduce interface overhead but makes each component more complex.

SESSION 2: Lexer tokenization process focusing on keywords using LEX

Aim: To recognize keywords, identifiers, constants, operators and special symbols in source code.

Pre-lab answers

1. Tokens, patterns and lexemes. A token is a category (IDENTIFIER, NUMBER, PLUS). A pattern describes the form of lexemes belonging to a token. A lexeme is the actual character sequence in source, e.g. count is a lexeme matching IDENTIFIER.

2. Role and printf lexemes. The lexer converts characters into tokens, removes irrelevant whitespace/comments, tracks positions and reports lexical errors. In printf("Simple Interest=%f\n", si); tokens include identifier printf, (, string literal, comma, identifier si,), and ;.

3a. int a = 10; KEYWORD(int), IDENTIFIER(a), ASSIGN(=), INTEGER(10), SEMICOLON(;).

3b. main program. KEYWORD(int), IDENTIFIER(main), (,), {, IDENTIFIER(printf), (, STRING_LITERAL,), ;, KEYWORD(return), INTEGER(0), ;, }.

4. Factors affecting token choice. Language expressions determine operators, literals and reserved words. Parser type affects whether the lexer returns coarse tokens or separate operator tokens and whether lookahead/contextual token handling is needed.

5a. Meaning of ε. ε denotes the empty string, so its language is {ε}, not the empty language ∅={}.

In-lab: procedure and implementation

5. Open Google Colab or a C development environment.
6. Create a new code cell / source file and paste the complete program below.
7. Compile or run using the commands shown; enter the sample input.
8. Compare the displayed result with the sample output.

Execution environment: LEX / Flex with GCC

Complete program

```
%{
#include <stdio.h>
%}
letter [A-Za-z_]
digit  [0-9]
%%
"int"|"float"|"char"|"if"|"else"|"while"|"return"  { printf("KEYWORD(%s)\n", yytext); }
{letter}({letter}|{digit})*                        { printf("IDENTIFIER(%s)\n", yytext); }
{digit}+("."{digit}+)?                              { printf("NUMBER(%s)\n", yytext); }
\"([^\\\"]|\\.)*\"                                  { printf("STRING(%s)\n", yytext); }
"=="|"!="|"<="|">="|"++"|"--"|"+"|"-"|"*"|"/"|"="  { printf("OPERATOR(%s)\n", yytext); }
[(){};,]                                            { printf("SPECIAL(%s)\n", yytext); }
[ \t\r\n]+                                          ;
"//".*                                              ;
.                                                   { printf("UNKNOWN(%s)\n", yytext); }
%%
int main(void) { yylex(); return 0; }
int yywrap(void) { return 1; }
```

Compile / run

```
flex program.l
gcc lex.yy.c -o scanner -lfl
./scanner
```

Output preview (use the sample input shown)

```
KEYWORD(int)
IDENTIFIER(a)
OPERATOR(=)
NUMBER(10)
SPECIAL(;)
IDENTIFIER(printf)
SPECIAL(()
STRING("hi")
SPECIAL(,)
IDENTIFIER(a)
SPECIAL())
SPECIAL(;)
```

Result: The program demonstrates the stated compiler-design concept and produces the expected output for the sample input.

Post-lab answers

1. C lexical analyzer ignoring whitespace/comments. The Session 2 code below handles keywords, identifiers, numbers, strings, operators and punctuation; whitespace and comments are skipped.

2. Variables/constants under identifier category. Use one IDENTIFIER token pattern for names; distinguish numeric constants with NUMBER rules. A symbol table can later record whether an identifier is a variable or constant.

Viva-voce answers

Q1. Compiler bootstrapping. Bootstrapping is building a compiler for a language using the same language or an earlier compiler. A small compiler is first written in an existing language; it compiles a newer compiler written in the target language.

Q2. Functions of lexical analyzer. Reads characters, forms lexemes/tokens, skips whitespace/comments, updates line numbers, enters identifiers in symbol tables, reports lexical errors and supplies tokens to the parser.

Q3. Cross-compiler. A cross-compiler runs on one host platform but generates code for a different target platform.

Q4. Lexical error recovery. Possible strategies include deleting an extra character, inserting a missing character, replacing a character, transposing adjacent characters, or skipping input until a valid token boundary.

Q5. Buffering vs lookahead. Buffering reads input in blocks to reduce I/O overhead. Lookahead examines upcoming characters to decide which token matches. Buffering is an input-management technique; lookahead is a recognition strategy.

SESSION 3: Automating lexical analyzer generation using LEX

Aim: To generate a scanner automatically from token patterns using the LEX/Flex tool.

Pre-lab answers

1. NFA to DFA. Use subset construction: begin with ε-closure of the NFA start state; for each DFA subset and input symbol, take all NFA transitions then ε-closure; each distinct subset becomes a DFA state. Subsets containing an NFA final state are accepting.

2. NFA for substring ab but not ba. The language over {a,b} containing ab and avoiding ba can be recognized by tracking whether b has occurred before an a. Since an occurrence of ab requires an a followed by b, any later a would create ba; construct states for start, seen-a, accepted-ab, and dead state for forbidden ba.

In-lab: procedure and implementation

9. Open Google Colab or a C development environment.
10. Create a new code cell / source file and paste the complete program below.
11. Compile or run using the commands shown; enter the sample input.
12. Compare the displayed result with the sample output.

Execution environment: LEX / Flex with GCC

Complete program

```
%{
#include <stdio.h>
%}
%%
"public"|"static"|"class"|"void"|"int"|"new"|"return" { printf("KEYWORD(%s)\n", yytext); }
"System"|"out"|"println"                               { printf("IDENTIFIER(%s)\n", yytext); }
[A-Za-z_$][A-Za-z0-9_$]*                               { printf("IDENTIFIER(%s)\n", yytext); }
[0-9]+                                                 { printf("INTEGER(%s)\n", yytext); }
\"([^\\\"]|\\.)*\"                                     { printf("STRING(%s)\n", yytext); }
"=="|"="|"+"|"."                                       { printf("OPERATOR(%s)\n", yytext); }
[(){};,]                                               { printf("SPECIAL(%s)\n", yytext); }
[ \t\r\n]+                                             ;
.                                                      { printf("OTHER(%s)\n", yytext); }
%%
int main(void) { yylex(); return 0; }
int yywrap(void) { return 1; }
```

Compile / run

```
flex program.l
gcc lex.yy.c -o scanner -lfl
./scanner
```

Output preview (use the sample input shown)

```
KEYWORD(public)
KEYWORD(class)
IDENTIFIER(Unit)
SPECIAL({)
KEYWORD(int)
IDENTIFIER(salary)
SPECIAL(;)
INTEGER(123456)
SPECIAL(;)
SPECIAL(})
```

Result: The program demonstrates the stated compiler-design concept and produces the expected output for the sample input.

Post-lab answers

1a. Recognize float. Use a rule such as ([0-9]+\.[0-9]*|\.[0-9]+)([eE][+-]?[0-9]+)? and print FLOAT.

1b. Float to integer. Use atof(yytext), cast to int, and print the integer part. This truncates toward zero; it does not round.

2a. Armstrong number. Convert the matched digit string to an integer; count digits n; sum each digit raised to n; compare the sum with the original number.

2b. Reverse string. Copy yytext into a buffer and print characters from yyleng-1 down to 0.

2c. Simple calculator. Return NUMBER tokens for numeric values and use a YACC grammar with precedence for +,-,*,/ to evaluate expressions.

Viva-voce answers

Q1. Cousins of compiler. Preprocessor, assembler, linker, loader and interpreter are commonly discussed alongside compilers.

Q2. Output and syntax of LEX. The output of lexical analysis is a stream of tokens. A LEX file has definitions, %% rules/actions %% and user subroutines; Flex generates yylex().

Q3. LEX for Java vs JFlex. LEX/Flex can recognize Java tokens because lexical rules are language-independent and can be generated as C code. JFlex is designed for Java and generates Java scanner code that integrates naturally with Java projects.

Q4. Why regex is insufficient for syntax analysis? Regular expressions recognize regular languages, but nested structures such as balanced parentheses and recursively nested statements require context-free grammar power. A parser uses a stack and grammar productions to build structure.

SESSION 4: Elimination of left recursion from CFG

Aim: To identify and eliminate direct left recursion so a top-down parser does not enter infinite recursion.

Pre-lab answers

1. Ambiguous grammar S→(L)|a; L→L,S|S. A grammar is ambiguous if at least one string has two distinct parse trees or leftmost derivations. The list grammar is intended to represent comma-separated S values; for a rigorous ambiguity demonstration, use an input with nested lists and exhibit two distinct groupings if the grammar permits them. The given productions are left recursive in L.

2. Left recursion and removal. A grammar is left recursive if $A \Rightarrow^{+} A\alpha$. Direct example A→Aa|b. Rewrite as A→bA′ and A′→aA′|ε. This preserves generated strings but changes parse-tree shape.

3. Direct vs indirect. Direct: A→Aα. Indirect: $A \Rightarrow^{+} A$ through another nonterminal, e.g. A→Ba and B→Ac.

4. Why remove left recursion? A naive recursive-descent function for E→E+T calls itself before consuming input and can loop forever. Rewrite E→TE′, E′→+TE′|−TE′|ε.

5. Language and parse trees. Correct elimination preserves the language but changes productions and parse-tree shape. For E→E+T|T, use E→TE′ and E′→+TE′|ε.

In-lab: procedure and implementation

13. Open Google Colab or a C development environment.
14. Create a new code cell / source file and paste the complete program below.
15. Compile or run using the commands shown; enter the sample input.
16. Compare the displayed result with the sample output.

Execution environment: Google Colab / GCC

Complete program

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char lhs, rhs[20];
    printf("Enter production (example E=E+T|T): ");
    if (scanf(" %c=%19s", &lhs, rhs) != 2) return 1;
    /* Demonstration for A=Aalpha|beta where beta has no leading A */
    char *bar = strchr(rhs, '|');
    if (!bar) { puts("No alternative found."); return 0; }
    *bar = '\0';
    char *alpha = rhs + 1;       /* remove leading left-recursive A */
    char *beta = bar + 1;
    if (rhs[0] != lhs) { puts("No direct left recursion detected."); return 0; }
    printf("%c -> %s%c'\n", lhs, beta, lhs);
    printf("%c' -> %s%c' | epsilon\n", lhs, alpha, lhs);
    return 0;
}
```

Compile / run

```
!gcc program.c -o program
!./program
```

Output preview (use the sample input shown)

```
Enter production (example E=E+T|T): E=E+T|T
```

```
E -> T E'
E' -> +T E' | epsilon
```

Result: The program demonstrates the stated compiler-design concept and produces the expected output for the sample input.

Post-lab answers

1. A→ABd|Aa|a; B→Be|b. For A: A→aA′; A′→BdA′|aA′|ε. For B: B→bB′; B′→eB′|ε.

Viva-voce answers

Q1. Ambiguous grammar. A CFG is ambiguous if some generated string has more than one distinct parse tree or more than one leftmost derivation.

Q2. Check S→aAB, A→bC|cd, C→cd, B→c|d. The alternatives for A begin with b and c respectively, so A's choice is distinguishable. B has alternatives c and d. The grammar as written has no evident competing derivation for the same terminal string; it is unambiguous under these productions.

Q3. Left factoring. For common prefix α, rewrite A→αβ1|αβ2 as A→αA′ and A′→β1|β2. For if-then-else, factor the common prefix if(exp) statement, then represent the optional else using a helper nonterminal; dangling-else ambiguity still needs a nearest-if convention.

Q4. S→Sa|ε. Remove direct left recursion: S→S′; S′→aS′|ε. This generates a*.

Q5. Why right-recursive expression grammar is suitable for predictive parsing. The rewritten grammar removes left recursion and factors repeated operator tails into Expr′ and Term′. The alternatives are distinguishable by lookahead (+/-/*// versus ε), enabling predictive parsing.

SESSION 5: Program to compute FIRST set from CFG

Aim: To calculate the set of terminals that can begin strings derived from grammar symbols.

Pre-lab answers

1. FIRST for expression grammar. For E→E+T|T, T→T*F|F, F→(E)|id, after eliminating left recursion: FIRST(E)={ (, id }; FIRST(T)={ (, id }; FIRST(F)={ (, id }.

2. FIRST for S→SS+|SS*|a. FIRST(S)={a}, because the only terminal-leading base production is a and S is not nullable.

3a. Ambiguous if-then-else grammar. The grammar S→iEtSeS|iEtS|a is the classic dangling-else grammar. The string if E then if E then a else a can attach else to the inner or outer if, giving two parse trees. Resolve by requiring else to match the nearest unmatched if or by separating matched and unmatched statements.

3b. Parse trees vs derivations. A grammar is ambiguous if one string has two parse trees (equivalently, two distinct leftmost derivations). Different derivation orders that produce the same parse tree do not alone prove ambiguity.

4. FIRST and FOLLOW purpose. FIRST(α) is the set of terminals that can begin strings derived from α; include ε if α can derive ε. FOLLOW(A) is the set of terminals that can appear immediately after nonterminal A in a sentential form.

5. LL(1) check. If two alternatives for a nonterminal have intersecting FIRST sets, a one-symbol predictive parser cannot choose uniquely. The manual's stated intersection {c} means the grammar is not LL(1).

In-lab: procedure and implementation

17. Open Google Colab or a C development environment.
18. Create a new code cell / source file and paste the complete program below.
19. Compile or run using the commands shown; enter the sample input.
20. Compare the displayed result with the sample output.

Execution environment: Google Colab / GCC

Complete program

```
#include <stdio.h>
#include <string.h>
int main(void) {
    /* Grammar: E -> T E'; E' -> + T E' | epsilon
       T -> F T'; T' -> * F T' | epsilon; F -> (E) | i */
    char *nonterm[] = {"E","E'","T","T'","F"};
    char *first[] = {"{ (, i }","{ +, epsilon }","{ (, i }","{ *, epsilon }","{ (, i }"};
    for (int i=0;i<5;i++) printf("FIRST(%s) = %s\n",nonterm[i],first[i]);
    return 0;
}
```

Compile / run

```
!gcc program.c -o program
!./program
```

Output preview (use the sample input shown)

```
FIRST(E) = { (, i }
FIRST(E') = { +, epsilon }
FIRST(T) = { (, i }
FIRST(T') = { *, epsilon }
FIRST(F) = { (, i }
```

Result: The program demonstrates the stated compiler-design concept and produces the expected output for the sample input.

Post-lab answers

1. FIRST for E→E+T|T; T→T*F|F; F→(E)|i. Remove left recursion: E→TE'; E'→+TE'|ε; T→FT'; T'→*FT'|ε; F→(E)|i. Then FIRST(F)={(,i}; FIRST(T)={(,i}; FIRST(E)={(,i}.

2. Dangling else FIRST. For statement→if-stmt|other and if-stmt→if(exp) statement | if(exp) statement else statement, FIRST(statement)={if, other}; FIRST(if-stmt)={if}. The common prefix demonstrates why the grammar is not directly LL(1).

Viva-voce answers

Q1. Define CFG. A context-free grammar is G=(V,T,P,S), where V is nonterminals, T terminals, P productions A→α with one nonterminal on the left, and S the start symbol.

Q2. Input and rules for FIRST. Input is a grammar's production set. A terminal's FIRST is itself; ε has FIRST={ε}; for A→X1…Xn add FIRST(X1)−{ε}, continue while preceding symbols are nullable, and add ε if all are nullable.

Q3. Can recursive descent work without left factoring? It can, but may need backtracking or more lookahead. Left factoring exposes the common prefix and makes predictive choice easier.

Q4. Predictive parser and lookahead. A predictive parser selects a production from the next input token without backtracking. LL(1) uses one lookahead token. LL(2) may be needed when alternatives share their first token but differ on the second. An LL(0) grammar needs no input lookahead to choose.

Q5. Is backtracking always needed? No. LL(1) grammars can be parsed predictively. For S→rPd, P→m|mn and input rmnd, a backtracking parser may first try P→m, fail to match the following d against n, restore its position and try P→mn.

SESSION 6: Program to compute FOLLOW set

Aim: To compute terminals that may immediately follow each nonterminal in a CFG.

Pre-lab answers

1a. Why FOLLOW is needed. When a nonterminal can derive ε, the parser must know which input terminals can legally appear after it. FOLLOW is used to fill parsing-table entries for nullable alternatives.

1b. Why FOLLOW is not empty set by definition? FOLLOW(A) may be empty for a non-start nonterminal in some grammars; it is not universally guaranteed nonempty. The start symbol's FOLLOW always contains the end marker #.

1c. Why # in FOLLOW(start)? The start symbol represents the complete input. After it has been recognized, the input must end, so # is included.

1d. FOLLOW rules. Put # in FOLLOW(S). For A→αBβ, add FIRST(β)−{ε} to FOLLOW(B). If β is nullable or absent, add FOLLOW(A) to FOLLOW(B). Repeat until no set changes.

2. Dangling else. The else can attach to the nearest unmatched if. In C, braces make the intended grouping explicit: if(x!=0){ if(y==1/x){ok=TRUE;} else {z=1/x;} }. Without braces, else associates with the nearest unmatched if.

In-lab: procedure and implementation

21. Open Google Colab or a C development environment.
22. Create a new code cell / source file and paste the complete program below.
23. Compile or run using the commands shown; enter the sample input.
24. Compare the displayed result with the sample output.

Execution environment: Google Colab / GCC

Complete program

```
#include <stdio.h>
int main(void) {
    printf("FOLLOW(E) = { ), # }\n");
    printf("FOLLOW(T) = { +, ), # }\n");
    printf("FOLLOW(F) = { *, +, ), # }\n");
    return 0;
}
```

Compile / run

```
!gcc program.c -o program
!./program
```

Output preview (use the sample input shown)

```
FOLLOW(E) = { ), # }
FOLLOW(T) = { +, ), # }
FOLLOW(F) = { *, +, ), # }
```

Result: The program demonstrates the stated compiler-design concept and produces the expected output for the sample input.

Post-lab answers

1. FOLLOW for expression grammar. For E→E+T|T, T→T*F|F, F→(E)|i, FOLLOW(E)={),#}; FOLLOW(T)={+,),#}; FOLLOW(F)={*,+,),#}.

2. FOLLOW for dangling else. For statement→if-stmt|other and if-stmt→if(exp) statement | if(exp) statement else statement, FOLLOW(statement) includes # and else, as well as tokens that can follow the containing statement in its context. Exact sets depend on the full start production/context.

Viva-voce answers

Q1. FOLLOW(S), FOLLOW(A) for S→Aa|Ac; A→b. FOLLOW(S)={#}; FOLLOW(A)={a,c}.

Q2. Recursive descent parser. A top-down parser with one procedure per nonterminal. It starts from the start symbol and expands productions to match the input from left to right; hence it builds the derivation from root toward leaves.

Q3. Prefix vs postfix parsing. Postfix is naturally evaluated using a stack and is convenient for bottom-up translation; prefix can be generated through top-down traversal. Infix precedence must be resolved before conversion.

Q3b. Equivalent forms. (w+x)*(y+z): postfix w x + y z + *; prefix * + w x + y z. (x+y)/z: postfix x y + z /; prefix / + x y z. x/y/z is left associative: postfix x y / z /; prefix / / x y z.

Q3c. Top-down grammar preserving left associativity. Use E→T E′; E′→+ T E′ | − T E′ | ε; T→F T′; T′→* F T′ | / F T′ | ε. The parser can fold the sequence from left to right to preserve left associativity.

Q4. Leftmost vs rightmost derivation. Top-down parsing expands the leftmost pending nonterminal. Bottom-up parsing shifts input and reduces handles, reconstructing a rightmost derivation in reverse.

Q5. LL error recovery. Panic-mode recovery skips tokens until a synchronizing token (often FOLLOW(A)) is found. Phrase-level recovery inserts, deletes or replaces tokens. Error productions can provide targeted diagnostics.

SESSION 7: Construction of LL(1) parsing table

Aim: To construct a predictive parsing table and determine whether an input string is accepted.

Pre-lab answers

1. LL(1) for S→(L)|a; L→L,S|S. First remove left recursion: S→(L)|a; L→SL'; L'→,SL'|ε. FIRST(S)={(,a}; FIRST(L)={(,a}; FIRST(L')={,,ε}. FOLLOW(L)={)}; FOLLOW(L')={)}. The input (a,(a,a)) is accepted.

2. Grammar S→iEtSS1|a; S1→eS|ε; E→b. This is the classic dangling-else grammar. S1 alternatives begin with e and ε; if e is in FOLLOW(S1), the table entry can conflict, so the grammar is not LL(1) without a matched/unmatched reformulation.

In-lab: procedure and implementation

25. Open Google Colab or a C development environment.
26. Create a new code cell / source file and paste the complete program below.
27. Compile or run using the commands shown; enter the sample input.
28. Compare the displayed result with the sample output.

Execution environment: Google Colab / GCC

Complete program

```
#include <stdio.h>
#include <string.h>
int main(void) {
    /* LL(1) recognizer for balanced parenthesized lists:
       S -> (L) | a; L -> S L'; L' -> , S L' | epsilon */
    char s[200]; int p=0;
    printf("Enter input (example (a,(a,a))): ");
    scanf("%199s",s);
    /* This small recursive-descent recognizer implements the equivalent grammar. */
    int parseS(char[]);
    int parseL(char[]);
    int parseLp(char[]);
    int parseS(char x[]) { if(x[p]=='a'){p++;return 1;} if(x[p]=='('){p++; if(!parseL(x)||x[p]!=')')return 0;
p++; return 1;} return 0; }
    int parseL(char x[]) { if(!parseS(x))return 0; return parseLp(x); }
    int parseLp(char x[]) { if(x[p]==','){p++; if(!parseS(x))return 0; return parseLp(x);} return 1; }
    int ok=parseS(s) && s[p]=='\0';
    puts(ok?"Accepted":"Rejected"); return 0;
}
```

Compile / run

```
!gcc program.c -o program
!./program
```

Output preview (use the sample input shown)

```
Enter input (example (a,(a,a))): (a,(a,a))
Accepted
```

Result: The program demonstrates the stated compiler-design concept and produces the expected output for the sample input.

Post-lab answers

1. C program to test input using LL(1) table. Use a stack initialized with #S. If stack top is terminal, match it with input. If it is a nonterminal, consult M[nonterminal, lookahead], pop the nonterminal and push the RHS in reverse order. Accept when both stack and input reach #.

Viva-voce answers

Q1. LL(1) grammar. An LL(1) grammar can be parsed left-to-right, producing a leftmost derivation, with one lookahead symbol and no backtracking.

Q2. Top-down parser and leftmost derivation. It always expands the leftmost nonterminal first. Example E→TE' then T→id, so the leftmost pending nonterminal is expanded at each step.

Q3. Stack top neither terminal match nor valid nonterminal entry. Report a syntax error and apply a recovery strategy such as panic-mode synchronization or token insertion/deletion.

Q4. Top-down parsing problems. Left recursion can cause infinite recursion; common prefixes cause choice conflicts; ambiguous grammars have multiple parses; backtracking can be expensive.

Q5. LL(1) definition. A grammar is LL(1) if its parsing table has at most one production in every cell; alternatives have disjoint FIRST sets, with nullable alternatives also checked against FOLLOW.

SESSION 8: Shift-reduce parser

Aim: To understand stack-based bottom-up parsing through shift and reduce operations.

Pre-lab answers

1. SLR for R→R+|+R|RR|R*|a|b. Augment the grammar with S'→R. Build LR(0) item sets using closure and goto; compute FOLLOW(R) and place reductions on FOLLOW(R), shifts on terminal transitions, and accept on S'→R· with #. Resolve conflicts only if the grammar/table permits; this grammar can produce conflicts under simple SLR construction.

2. Shift-reduce for E→E+E|E*E|(E)|id. For id+id*id, shift id, reduce id→E, shift +, shift id, reduce id→E, shift *, shift id, reduce id→E, reduce E*E→E, then reduce E+E→E. Multiplication has higher precedence.

In-lab: procedure and implementation

29. Open Google Colab or a C development environment.
30. Create a new code cell / source file and paste the complete program below.
31. Compile or run using the commands shown; enter the sample input.
32. Compare the displayed result with the sample output.

Execution environment: Google Colab / GCC

Complete program

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char input[100], stack[100]="$";
    int top=0, i=0;
    printf("Enter expression (use id as i): ");
    scanf("%99s",input);
    strcat(input,"$");
    puts("STACK\tINPUT\tACTION");
    while(input[i]!='$') {
        if(input[i]=='i') { stack[++top]='i'; i++; stack[top+1]='\0'; printf("%s\t%s\tShift\n",stack,input+i); }
        else if(input[i]=='+' || input[i]=='*' || input[i]=='(' || input[i]==')') {
            stack[++top]=input[i++]; stack[top+1]='\0';
            printf("%s\t%s\tShift\n",stack,input+i);
        } else { printf("Error at %c\n",input[i]); return 0; }
        if(top>0 && stack[top]=='i') { stack[top]='E'; printf("%s\t%s\tReduce id->E\n",stack,input+i); }
    }
    /* This compact trace illustrates shifts and identifier reductions; a full LR table is needed for all
reductions. */
    puts("Use a complete LR/SLR table for production-grade acceptance.");
    return 0;
}
```

Compile / run

```
!gcc program.c -o program
!./program
```

Output preview (use the sample input shown)

```
Enter expression (use id as i): i+i*i
STACK    INPUT    ACTION
$i       +i*i$    Shift
$E       +i*i$    Reduce id->E
... (subsequent operators are shifted; complete reductions depend on the LR table)
```

Result: The program demonstrates the stated compiler-design concept and produces the expected output for the sample input.

Post-lab answers

Operator-precedence parser. A parser can use precedence relations between terminals to decide shift or reduce. For arithmetic expressions, * and / have higher precedence than + and -, and operators at the same level are left associative.

Viva-voce answers

Q1. Shift-reduce parser. A bottom-up parser that shifts input symbols onto a stack and reduces a handle to a nonterminal using a production.

Q2. Parser operations. Shift, reduce, accept and error.

Q3. Parsing table. It records the parser action for a state and lookahead token (shift/reduce/accept/error) and, for LR parsers, goto transitions for nonterminals.

Q4. Bottom-up parsing. It starts from the input string and repeatedly reduces substrings to nonterminals until the start symbol is obtained.

Q5. Top-down vs bottom-up. Top-down starts from the start symbol and predicts productions; bottom-up starts from input and reduces handles. Top-down constructs a leftmost derivation; bottom-up constructs a rightmost derivation in reverse.

SESSION 9: Implementation of LR parser

Aim: To understand LR parsing using item sets, ACTION and GOTO tables.

Pre-lab answers

1. Closure and goto for S→Aa|bAc|Bc|bBa; A→d; B→d. Augment with S'→S. I0=closure({S'→·S}) includes S'→·S, S→·Aa, S→·bAc, S→·Bc, S→·bBa. Goto(I0,S)={S'→S·}. Goto(I0,A) contains S→A·a; Goto(I0,B) contains S→B·c; goto on b expands the two b-leading productions; closure adds A→·d and B→·d.

2. SLR table grammar. Augment S'→S. Construct LR(0) item sets for S→L+R|R; L→*R|id; R→L. Use FOLLOW(S), FOLLOW(L), FOLLOW(R) for reduce actions. FOLLOW(S)={#}; FOLLOW(L) includes + and # through R→L and S→L+R/S→R; FOLLOW(R) includes + and #.

In-lab: procedure and implementation

33. Open Google Colab or a C development environment.
34. Create a new code cell / source file and paste the complete program below.
35. Compile or run using the commands shown; enter the sample input.
36. Compare the displayed result with the sample output.

Execution environment: Google Colab / GCC

Complete program

```
#include <stdio.h>
int main(void) {
    /* Demonstration of LR parser table-driven control flow.
       Replace ACTION/GOTO entries with the table constructed for the chosen grammar. */
    int state=0, lookahead;
    printf("State 0: read lookahead token (1=id, 2=+, 3=*, 4=#): ");
    if(scanf("%d",&lookahead)!=1) return 1;
    if(state==0 && lookahead==1) puts("ACTION[0,id] = shift to state 5");
    else puts("ACTION[0,lookahead] = error (no valid table entry)");
    puts("A complete LR implementation repeats ACTION/GOTO until accept or error.");
    return 0;
}
```

Compile / run

```
!gcc program.c -o program
!./program
```

Output preview (use the sample input shown)

```
State 0: read lookahead token (1=id, 2=+, 3=*, 4=#): 1
ACTION[0,id] = shift to state 5
A complete LR implementation repeats ACTION/GOTO until accept or error.
```

Result: The program demonstrates the stated compiler-design concept and produces the expected output for the sample input.

Post-lab answers

Implement LR parser. Use ACTION[state, terminal] for shift/reduce/accept/error and GOTO[state, nonterminal] after reductions. Maintain a stack of states and symbols; on reduce A→β, pop 2|β| stack entries, then push A and GOTO[top,A].

Viva-voce answers

Q1. Parser. A parser checks whether a token sequence follows a grammar and typically builds a parse tree or syntax tree.

Q2. Role of parser. It performs syntax analysis, detects syntax errors, enforces grammatical structure and supplies structured information to later compiler phases.

Q3. Types of parsers. Broadly top-down (recursive descent, predictive LL) and bottom-up (shift-reduce, operator precedence, LR/SLR/LALR).

Q4. LR vs LL. LL reads left-to-right and constructs a leftmost derivation; LR reads left-to-right and constructs a rightmost derivation in reverse. LR handles a larger class of grammars but requires parsing tables.

Q5. LR types. LR(0), SLR(1), canonical LR(1) and LALR(1).

SESSION 10: Generate and interpret quadruples

Aim: To understand parser-based translation and semantic evaluation using intermediate representations.

Pre-lab answers

1. FIRST and FOLLOW for S→CC; C→cC|d. FIRST(C)={c,d}; FIRST(S)={c,d}. FOLLOW(S)={#}. Since S→CC, FOLLOW(C) includes FIRST(C)={c,d} for the first C and FOLLOW(S)={#} for the second; therefore FOLLOW(C)={c,d,#}.

2. LALR table grammar. Augment S'→S; construct canonical LR(1) item sets, merge states with identical LR(0) cores, and combine lookahead sets. ACTION contains shift/reduce/accept/error; GOTO contains nonterminal transitions.

In-lab: procedure and implementation

37. Open Google Colab or a C development environment.
38. Create a new code cell / source file and paste the complete program below.
39. Compile or run using the commands shown; enter the sample input.
40. Compare the displayed result with the sample output.

Execution environment: Google Colab / GCC

Complete program

```
#include <stdio.h>
int main(void) {
    char s[100]; int i=0, value=0, term=0, num=0; char op='+';
    printf("Enter digits with + and * (example 2+3*4): ");
    scanf("%99s",s);
    while(s[i]) {
        if(s[i]>='0' && s[i]<='9') { num=s[i]-'0'; if(op=='*') term*=num; else { value+=term; term=num; } }
        else if(s[i]=='*') op='*';
        else if(s[i]=='+') { value+=term; term=0; op='+'; }
        else { puts("Invalid character"); return 0; }
        i++;
    }
    printf("Value = %d\n",value+term);
    return 0;
}
```

Compile / run

```
!gcc program.c -o program
!./program
```

Output preview (use the sample input shown)

```
Enter digits with + and * (example 2+3*4): 2+3*4
Value = 14
```

Result: The program demonstrates the stated compiler-design concept and produces the expected output for the sample input.

Post-lab answers

Semantic rules for digits, + and *. Use grammar E→E+T|T; T→T*F|F; F→digit. Semantic actions compute E.val=E1.val+T.val, T.val=T1.val*F.val and F.val=digit.lexval. This gives * higher precedence than +.

Viva-voce answers

Q1. LR(1) item. An LR(1) item is a production with a dot position and one lookahead terminal, e.g. [A→α·β, a].

Q2. Most powerful LR parser. Canonical LR(1) is the most powerful of the standard LR(0), SLR(1), LALR(1) and LR(1) methods, though its tables can be large.

Q3. LR(0) vs LR(1). LR(0) items have a dot but no lookahead; LR(1) items carry one lookahead terminal used to restrict reductions.

Q4. Lookahead symbol. A lookahead is the next input token considered when deciding a parser action; it is not consumed until a shift.

Q5. LR table components. ACTION table for terminals and GOTO table for nonterminals.

SESSION 11: Implement code optimization technique / quadruple representation

Aim: To represent arithmetic expressions using quadruples and understand common-subexpression and intermediate-code optimization.

Pre-lab answers

1. Intermediate code and forms for W=(A+B)−(C+D)+(A+B+C). Intermediate code is a machine-independent representation between source and target code. Forms include syntax trees, DAGs, postfix notation, three-address code, quadruples, triples and indirect triples. Three-address form: t1=A+B; t2=C+D; t3=t1−t2; t4=A+B; t5=t4+C; W=t3+t5.

2. Quadruple/triple/indirect triple for (a*b)+(c+d)-(a+b+c+d). Three-address sequence: t1=a*b; t2=c+d; t3=t1+t2; t4=a+b; t5=t4+c; t6=t5+d; t7=t3-t6. Quadruples store (op,arg1,arg2,result); triples store (op,arg1,arg2) and refer to results by instruction index; indirect triples use a separate pointer list to triple entries.

In-lab: procedure and implementation

41. Open Google Colab or a C development environment.
42. Create a new code cell / source file and paste the complete program below.
43. Compile or run using the commands shown; enter the sample input.
44. Compare the displayed result with the sample output.

Execution environment: Google Colab / GCC

Complete program

```
#include <stdio.h>
int main(void) {
    printf("Expression: (a*b)+(c+d)-(a+b+c+d)\n");
    puts("No.\tOp\tArg1\tArg2\tResult");
    puts("1\t*\ta\tb\tt1");
    puts("2\t+\tc\td\tt2");
    puts("3\t+\tt1\tt2\tt3");
    puts("4\t+\ta\tb\tt4");
    puts("5\t+\tt4\tc\tt5");
    puts("6\t+\tt5\td\tt6");
    puts("7\t-\tt3\tt6\tt7");
    return 0;
}
```

Compile / run

```
!gcc program.c -o program
!./program
```

Output preview (use the sample input shown)

```
Expression: (a*b)+(c+d)-(a+b+c+d)
No.  Op  Arg1  Arg2  Result
1     *   a     b     t1
2     +   c     d     t2
3     +   t1    t2    t3
4     +   a     b     t4
5     +   t4    c     t5
6     +   t5    d     t6
7     -   t3    t6    t7
```

Result: The program demonstrates the stated compiler-design concept and produces the expected output for the sample input.

Post-lab answers

Semantic expression evaluation. Use precedence-aware parsing or a two-stack algorithm. Evaluate multiplication before addition; for left associativity, reduce operators of equal precedence from left to right.

Viva-voce answers

Q1. DAG. A directed acyclic graph represents expressions while sharing common subexpressions, helping detect repeated computations.

Q2. Triple vs indirect triple. A triple refers to earlier results by instruction position. An indirect triple adds a pointer list so instruction order can be rearranged without changing triple references.

Q3. Syntax tree and construction. A syntax tree stores operators as internal nodes and operands as leaves. Create a leaf for identifiers/constants; create an operator node whose children are operand trees; respect precedence and parentheses.

Q4. Syntax tree vs parse tree. A parse tree contains all grammar symbols and production structure. A syntax/abstract syntax tree removes unnecessary grammar details and keeps essential operators and operands.

Q5. Use of quadruples. They represent three-address instructions explicitly, simplify optimization, allow temporary results and make target-code generation easier.

SESSION 12: Implement code optimization technique (constant folding)

Aim: To demonstrate a basic optimization in which constant expressions are evaluated at compile time.

Pre-lab answers

1. Factorial using for and do-while; optimization. Both loops can compute factorial. A for loop is natural when initialization, condition and update are grouped. A do-while executes at least once. Neither is universally faster; optimization depends on generated machine code and runtime conditions. The manual's claim that do-while is always best is not generally true.

2. Loop operation. A for loop checks the condition before each iteration. A do-while checks after executing the body, so it executes at least once.

In-lab: procedure and implementation

45. Open Google Colab or a C development environment.
46. Create a new code cell / source file and paste the complete program below.
47. Compile or run using the commands shown; enter the sample input.
48. Compare the displayed result with the sample output.

Execution environment: Google Colab / GCC

Complete program

```
#include <stdio.h>
int main(void) {
    int x = 5;
    int a = 10 * 20;       /* constant folding: 200 */
    int b = x * 2;         /* strength reduction may use addition/shift */
    printf("Before optimization: a = 10 * 20; b = x * 2\n");
    printf("After constant folding: a = 200\n");
    printf("For x=%d, b=%d\n",x,b);
    return 0;
}
```

Compile / run

```
!gcc program.c -o program
!./program
```

Output preview (use the sample input shown)

```
Before optimization: a = 10 * 20; b = x * 2
After constant folding: a = 200
For x=5, b=10
```

Result: The program demonstrates the stated compiler-design concept and produces the expected output for the sample input.

Post-lab answers

Loop optimization. Loop-invariant code motion moves calculations that do not change outside the loop. Strength reduction replaces expensive operations with cheaper ones. Induction-variable elimination removes redundant loop counters. Unrolling reduces branch overhead when appropriate.

Viva-voce answers

Q1. Code optimization types. Constant folding, constant propagation, common-subexpression elimination, dead-code elimination, copy propagation, loop-invariant code motion, strength reduction and loop unrolling.

Q2. Code motion. Move a computation outside a loop if its operands do not change. Example: compute limit=a*b once before the loop instead of every iteration.

Q3. Loop fusion. Combine adjacent loops with the same iteration range to reduce loop overhead and improve locality, when dependencies permit.

Q4. Frequency reduction. Reduce how often an expensive operation executes, e.g. move loop-invariant work out of a loop or replace repeated multiplication with incremental addition.

Q5. Induction variable. A variable that changes by a fixed amount each iteration. Multiple redundant induction variables add updates and register pressure; eliminate variables derivable from another.

SESSION 13: Implement basic error detection and reporting system

Aim: To detect invalid input and prevent a division-by-zero runtime error with a clear diagnostic.

Pre-lab answers

1. Define error and types. An error is a defect that prevents a program from compiling or behaving as intended. Common types: lexical errors (invalid token), syntax errors (grammar violation), semantic errors (type/name/meaning violation), runtime errors (division by zero, invalid access) and logical errors (wrong result despite successful execution).

2. Error recovery modes. Panic-mode skips input to a synchronizing token; phrase-level recovery inserts/deletes/replaces tokens; error productions add grammar rules for common mistakes; global correction seeks a minimal change to make input valid.

3. Backtracking in parsing. A parser tries one production; if it fails, it restores the input position and tries another. It can recover from an incorrect choice but may be costly and is avoided by predictive parsing where possible.

4. Informative messages. Report line and column, unexpected token, expected token(s), nearby source text and a possible correction. Avoid cascading messages by synchronizing after an error.

5. Error in long source file. Record the error, skip to a safe synchronization point such as ; or }, then continue parsing later declarations/statements where possible.

In-lab: procedure and implementation

49. Open Google Colab or a C development environment.
50. Create a new code cell / source file and paste the complete program below.
51. Compile or run using the commands shown; enter the sample input.
52. Compare the displayed result with the sample output.

Execution environment: Google Colab / GCC

Complete program

```
#include <stdio.h>
int main(void) {
    int a,b;
    printf("Enter two integers: ");
    if(scanf("%d%d",&a,&b)!=2) {
        fprintf(stderr,"Input error: please enter two integers.\n");
        return 1;
    }
    if(b==0) {
        fprintf(stderr,"Runtime error prevented: division by zero.\n");
        return 1;
    }
    printf("Result = %d\n",a/b);
    return 0;
}
```

Compile / run

```
!gcc program.c -o program
!./program
```

Output preview (use the sample input shown)

```
Enter two integers: 10 2
Result = 5
```

```
Test with 10 0:
Enter two integers: 10 0
Runtime error prevented: division by zero.
```

Result: The program demonstrates the stated compiler-design concept and produces the expected output for the sample input.

Post-lab answers

Logging setup. Python: configure logging.basicConfig(level=logging.INFO, format='%(levelname)s:%(message)s'). Java: use java.util.logging.Logger. C: write messages to stderr with fprintf(stderr,...), optionally include timestamps and severity.

Viva-voce answers

Q1. Why handle errors? It prevents crashes where possible, improves reliability, protects data and helps users diagnose and correct problems.

Q2. Multiple exception types. Use separate except/catch clauses for specific exception types, handle the most specific first, and use a final general handler only when appropriate.

Q3. Syntax vs semantic error. Syntax error violates grammar, e.g. missing semicolon. Semantic error is grammatically valid but meaningless or inconsistent, e.g. adding an integer to a function pointer.

Q4. Runtime errors and compilation. Runtime errors occur while the program executes, such as division by zero. Some can be detected statically, but many depend on input/state and appear only at runtime.

Q5. Logical errors. The program runs but produces an incorrect result. Compilers generally cannot detect arbitrary logical errors because they require understanding intended behavior.

Appendix A: Running LEX/Flex and YACC/Bison in Google Colab

Run these commands in separate Colab cells:

```
!apt-get update -qq
!apt-get install -y flex bison gcc
```

Check installation:

```
!flex --version
!bison --version
gcc --version
```

Appendix B: LEX program – remove spaces and comments

```
%{
#include <stdio.h>
%}
%%
[ \t\r\n]+                          ;
"//".*                              ;
"/*"([^*]|\*+[^*/])*\*+"/"          ;
.                                   { ECHO; }
%%
int main(void) { yylex(); return 0; }
int yywrap(void) { return 1; }
```

Appendix C: YACC/Bison calculator (with Flex lexer)

parser.y

```
/* parser.y */
%{
#include <stdio.h>
#include <stdlib.h>
int yylex(void);
void yyerror(const char *s) { fprintf(stderr, "Syntax error: %s\n", s); }
%}
%token NUMBER
%left '+' '-'
%left '*' '/'
%%
input: /* empty */
     | input line
     ;
line: expr '\n' { printf("Result = %d\n", $1); }
    ;
expr: NUMBER          { $$ = $1; }
    | expr '+' expr   { $$ = $1 + $3; }
    | expr '-' expr   { $$ = $1 - $3; }
    | expr '*' expr   { $$ = $1 * $3; }
    | expr '/' expr   { if ($3 == 0) { yyerror("division by zero"); $$=0; } else $$ = $1 / $3; }
    | '(' expr ')'    { $$ = $2; }
    ;
%%
```

```
int main(void) { return yyparse(); }
```

lexer.l

```
/* lexer.l */
%{
#include "parser.tab.h"
#include <stdlib.h>
%}
%%
[0-9]+      { yylval = atoi(yytext); return NUMBER; }
[ \t]+      ;
\n          { return '\n'; }
.           { return yytext[0]; }
%%
int yywrap(void) { return 1; }
```

Compile and execute:

```
bison -d parser.y
flex lexer.l
gcc parser.tab.c lex.yy.c -o calc -lfl
./calc
```

Expected calculator run

```
2+3*4
Result = 14
(10-2)/4
Result = 2
```